



DDA 3005 — Numerical Methods

The Inexact Newton Method

Lecture 25

December 12th

Andre Milzarek

SDS / CUHK-SZ

Repetition

Newton's Method in $\mathbb{R}^n$

- ▶ We can solve a system of nonlinear equations $F(\mathbf{x}) = \mathbf{0}$ via Newton's method:

$$\mathbf{x}^{k+1} = \mathbf{x}^k - DF(\mathbf{x}^k)^{-1}F(\mathbf{x}^k).$$

- ▶ This method converges quadratically to a solution $\mathbf{x}^*$ if $DF(\mathbf{x}^*)$ is invertible and $\mathbf{x}^0$ is chosen sufficiently close to $\mathbf{x}^*$.
 - ▶ Newton's method can be **globalized** by checking whether $\mathbf{x}^{k+1}$ decreases the **merit function** $m(\mathbf{x}) = \frac{1}{2}\|F(\mathbf{x})\|^2$.
 - ▶ The direction $\mathbf{d}^k = -DF(\mathbf{x}^k)^{-1}F(\mathbf{x}^k)$ is a **descent direction** for m .
- $\leadsto$ We can perform linesearch / backtracking on m to damp the Newton step $\mathbf{x}^k + \alpha_k \mathbf{d}^k$.

Application: Constrained Optimization & KKT Conditions

Equality Constrained Optimization

We consider and discuss nonlinear programs with equality constraints:

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad \text{s.t.} \quad h(\mathbf{x}) = \mathbf{0}.$$

where $f : \mathbb{R}^n \rightarrow \mathbb{R}$ and $h : \mathbb{R}^n \rightarrow \mathbb{R}^p$ are given.

Let $\mathbf{x}^*$ be a local solution of this problem. Then, according to the **KKT-conditions** there exists $\boldsymbol{\mu}^* \in \mathbb{R}^p$:

$$\nabla_{\mathbf{x}} L(\mathbf{x}^*, \boldsymbol{\mu}^*) = \nabla f(\mathbf{x}^*) + \nabla h(\mathbf{x}^*) \boldsymbol{\mu}^* = \mathbf{0} \quad \text{and} \quad h(\mathbf{x}^*) = \mathbf{0}.$$

- The KKT-conditions define a **nonlinear system of equations** with $n + p$ unknowns and $n + p$ equations.
- ↪ We can apply **Newton's method** to determine a solution of the nonlinear equation:

$$F(\mathbf{x}, \boldsymbol{\mu}) := \begin{bmatrix} \nabla_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\mu}) \\ h(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \mathbf{0} \\ \mathbf{0} \end{bmatrix}.$$

Generating Lagrange-Newton Steps

Let $(\mathbf{x}^k, \boldsymbol{\mu}^k)$ be the current iterate. Then, **Newton's step** $\mathbf{d}^k$ for the equation $F(\mathbf{x}, \boldsymbol{\mu}) = 0$ is given by

$$DF(\mathbf{x}^k, \boldsymbol{\mu}^k)\mathbf{d}^k = -F(\mathbf{x}^k, \boldsymbol{\mu}^k).$$

In particular, we can calculate

$$DF(\mathbf{x}, \boldsymbol{\mu}) = \begin{pmatrix} \nabla_{xx}^2 L(\mathbf{x}, \boldsymbol{\mu}) & \nabla_{\mu x}^2 L(\mathbf{x}, \boldsymbol{\mu}) \\ \nabla h(\mathbf{x})^\top & \mathbf{0} \end{pmatrix} = \begin{pmatrix} \nabla_{xx}^2 L(\mathbf{x}, \boldsymbol{\mu}) & \nabla h(\mathbf{x}) \\ \nabla h(\mathbf{x})^\top & \mathbf{0} \end{pmatrix}$$

and the **complete Newton update** simplifies to:

$$\begin{pmatrix} \nabla_{xx}^2 L(\mathbf{x}^k, \boldsymbol{\mu}^k) & \nabla h(\mathbf{x}^k) \\ \nabla h(\mathbf{x}^k)^\top & \mathbf{0} \end{pmatrix} \begin{pmatrix} \mathbf{d}_x^k \\ \mathbf{d}_\mu^k \end{pmatrix} = \begin{pmatrix} -\nabla_x L(\mathbf{x}^k, \boldsymbol{\mu}^k) \\ -h(\mathbf{x}^k) \end{pmatrix}$$

and $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{d}_x^k$, $\boldsymbol{\mu}^{k+1} = \boldsymbol{\mu}^k + \alpha_k \mathbf{d}_\mu^k$.

$\rightsquigarrow$ This special version is known as the **Lagrange-Newton method**.

Numerical Experiment

We consider the problem

$$\min_{\mathbf{x}} f(\mathbf{x}) = (x_1^2 - 1)(x_1^2 - 9) + 8x_1x_2 + (x_2^2 - 1)(x_2^2 - 9)$$

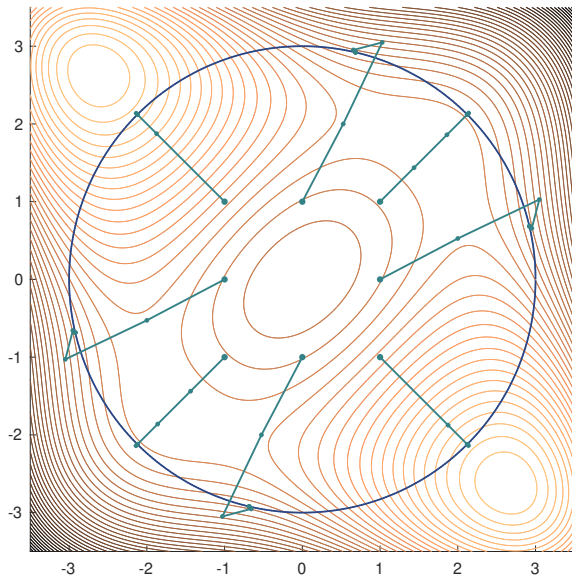
subject to $\|\mathbf{x}\| = 3$. Setting $h(\mathbf{x}) := \frac{1}{2}\|\mathbf{x}\|^2 - \frac{9}{2}$, we then have

$$\nabla_{\mathbf{x}} L(\mathbf{x}, \mu) = \begin{bmatrix} 4x_1(x_1^2 - 5) + 8x_2 \\ 4x_2(x_2^2 - 5) + 8x_1 \end{bmatrix} + \mu \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

Remarks:

- ▶ We implement a standard backtracking strategy for m .
- ▶ We test eight different initial points with $x_i^0 \in \{-1, 0, 1\}$ and $\mu = 1$.
- ▶ Due to the small size of the problem, we can use MATLAB's backslash or `linalg.solve` to solve the Newton systems.
- ▶ Newton's method converges quadratically within 6–7 steps.

Solution Paths



Inexact Newton Steps

Inexact Newton Steps

It is also possible to solve the linear system

$$DF(\mathbf{x}^k) \cdot \mathbf{d}^k = -F(\mathbf{x}^k)$$

only **approximately** up to a certain accuracy and to generate **inexact** and **cheaper Newton steps**.

Strategy: Find an approximate direction $\mathbf{d}^k$ such that:

$$\|DF(\mathbf{x}^k)\mathbf{d}^k + F(\mathbf{x}^k)\| \leq \text{tol}, \quad \text{tol} \geq 0.$$

Observations:

- ▶ If $DF(\mathbf{x}^k)$ is symmetric, we can generally apply the **CG-method** to obtain such an approximate direction.
- ↪ This is especially effective in the optimization case when $F = \nabla f$ and $DF = \nabla^2 f$ and when n is large!

Inexact Newton Steps: Discussion

Why does this work? ...

- ▶ Adaptive choices of tol , i.e.,

$$\text{tol} \equiv \text{tol}_k = \rho_k \|F(\mathbf{x}^k)\|, \quad \rho_k \rightarrow 0$$

allow to ensure fast local convergence!

- ▶ **Interpretation:** Closer to a solution / stationary point of the problem, we compute the Newton step with higher accuracy.
- ↪ There are many more techniques and options to boost the performance of Newton's method in large-scale settings!

Optimization: The Newton-CG Method

Let us consider the classical Newton method for the problem $\min_{\mathbf{x}} f(\mathbf{x})$. Then, we have

$$F(\mathbf{x}^k) = \nabla f(\mathbf{x}^k), \quad DF(\mathbf{x}^k) = \nabla^2 f(\mathbf{x}^k),$$

and the Newton direction is given by $\nabla^2 f(\mathbf{x}^k) \mathbf{d}^k = -\nabla f(\mathbf{x}^k)$.

Strategy: Since $\mathbf{A} = \nabla^2 f(\mathbf{x}^k)$ is symmetric, we can solve the Newton equation approximately via CG!

Line Search Newton-CG:

- In order to ensure well-definedness of the CG-method, one typically adds the safeguard:

$$(\mathbf{p}^k)^\top \mathbf{A} \mathbf{p}^k \leq 0 \quad \implies \quad \text{break.}$$

- Setting $\mathbf{x}^0 = \mathbf{0}$ (in CG), $\mathbf{b} = -\nabla f(\mathbf{x}^k)$ and $\mathbf{A} = \nabla^2 f(\mathbf{x}^k)$, the CG-method returns a descent direction $\mathbf{d}$ with

$$\nabla f(\mathbf{x}^k)^\top \mathbf{d} < 0.$$

↪ This can be utilized to achieve global convergence.

Additional Remarks and Discussion

Advantages:

- ▶ Storage of the full Hessian matrix $\nabla^2 f(\mathbf{x}^k)$ is not necessary.
- ▶ The CG-method only requires “Hessian $\times$ vector” applications, i.e., it only uses terms of the form $\nabla^2 f(\mathbf{x}^k) \cdot \mathbf{v}$.
- ▶ The combination of the Newton and CG-method is known as **line search Newton-CG method** or **truncated Newton method**.

Convergence:

- ▶ The combination of the CG-method and the classical Newton method is an example of an inexact Newton method.
- ▶ Fast local convergence can still be guaranteed!

The Full Newton-CG Method

Truncated Newton Method

1. Select $\mathbf{x}^0 \in \mathbb{R}^n$ and choose $\sigma, \gamma \in (0, 1)$, and $\{\rho_k\}_k$.

For $k = 0, 1, \dots$:

2. If $\|\nabla f(\mathbf{x}^k)\| \leq \varepsilon$, then STOP and $\mathbf{x}^k$ is the output.
3. Set $\mathbf{A} = \nabla^2 f(\mathbf{x}^k)$, $\mathbf{v}^0 = 0$, $\mathbf{r}^0 = \nabla f(\mathbf{x}^k)$, $\mathbf{p}^0 = -\mathbf{r}^0$, and $\text{tol} = \rho_k \cdot \|\nabla f(\mathbf{x}^k)\|$.
4. **For** $j = 0, 1, \dots$:
 - ▶ If $(\mathbf{p}^j)^\top \mathbf{A} \mathbf{p}^j \leq 0$ return $\mathbf{d}^k = \mathbf{v}^j$ (or $\mathbf{d}^k = -\nabla f(\mathbf{x}^k)$ if $j = 0$).
 - ▶ Compute $\sigma_j = \frac{\|\mathbf{r}^j\|^2}{(\mathbf{p}^j)^\top \mathbf{A} \mathbf{p}^j}$, $\mathbf{v}^{j+1} = \mathbf{v}^j + \sigma_j \mathbf{p}^j$, $\mathbf{r}^{j+1} = \mathbf{r}^j + \sigma_j \mathbf{A} \mathbf{p}^j$.
 - ▶ If $\|\mathbf{r}^{j+1}\| \leq \text{tol}$, then STOP and return $\mathbf{d}^k = \mathbf{v}^{j+1}$.
 - ▶ Calculate $\beta_{j+1} = \frac{\|\mathbf{r}^{j+1}\|^2}{\|\mathbf{r}^j\|^2}$ and $\mathbf{p}^{j+1} = -\mathbf{r}^{j+1} + \beta_{j+1} \mathbf{p}^j$.
5. Calculate α_k via backtracking and set $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{d}^k$.

Case Study: Logistic Regression

Application: Logistic Regression

Binary Classification:

- ▶ Let $\mathbf{a}_i \in \mathbb{R}^n$ $i = 1, \dots, m$ be data points with labels $b_i \in \{-1, 1\}$.
- ▶ The labels b_i indicate that the data points belong to two different classes $\mathcal{C}_1 \equiv +1$ and $\mathcal{C}_2 \equiv -1$.
- ▶ We want to **learn a function** $\ell : \mathbb{R}^n \rightarrow \{\pm 1\}$ that allows to **predict** whether a new point $\mathbf{a} \notin \{\mathbf{a}_1, \dots, \mathbf{a}_m\}$ belongs to class $\mathcal{C}_1$ or $\mathcal{C}_2$.

In **logistic regression**, we consider the optimization model:

$$\min_{\mathbf{x}, y} \frac{1}{m} \sum_{i=1}^m \log(1 + \exp(-b_i \cdot (\mathbf{a}_i^\top \mathbf{x} + y))) + \frac{\lambda}{2} \|\mathbf{x}\|^2, \quad \lambda \geq 0. \quad (1)$$

- ▶ In contrast to support vector machines, this model is based on a **probabilistic idea**.

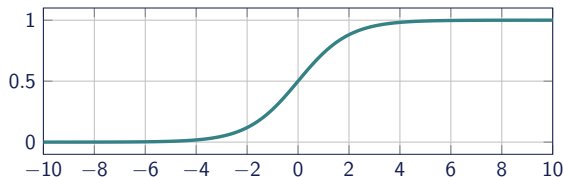
The Logistic Regression Model

- ▶ We try to find a good parametric model of the probability that a point $\mathbf{a}$ belongs to the class $\mathcal{C}_1$.
- ▶ In logistic regression, the probability model is based on the **sigmoid function**:

$$\sigma : \mathbb{R} \rightarrow \mathbb{R}, \quad \sigma(t) = \frac{1}{1 + \exp(-t)}.$$

Idea: We train a linear model $h(\mathbf{a}) = \mathbf{a}^\top \mathbf{x} + y$ with parameters $\mathbf{x}$ and y such that:

$$\sigma(h(\mathbf{a}_i)) \approx \begin{cases} 1 & \text{if } \mathbf{a}_i \text{ belongs to } \mathcal{C}_1, \text{ i.e., } b_i = +1 \\ 0 & \text{if } \mathbf{a}_i \text{ belongs to } \mathcal{C}_2, \text{ i.e., } b_i = -1, \end{cases} \quad \forall i.$$



Plot of σ

The Logistic Regression Model: Continued

- ▶ The parameters $\mathbf{x}$ and y are estimated using a **maximum (log)-likelihood estimate**.
- ▶ This yields the optimization model (1) (with $\lambda = 0$).
- ▶ After determining $(\mathbf{x}, y)$, a new data point $\mathbf{a} \in \mathbb{R}^n$ can be classified via:

$$\begin{cases} +1 & \text{if } \sigma(h(\mathbf{a})) > \frac{1}{2} \\ -1 & \text{if } \sigma(h(\mathbf{a})) \leq \frac{1}{2} \end{cases} \quad \text{or} \quad \begin{cases} \mathcal{C}_1 & \text{if } \sigma(h(\mathbf{a})) > \frac{1}{2} \\ \mathcal{C}_2 & \text{if } \sigma(h(\mathbf{a})) \leq \frac{1}{2} \end{cases}$$

Numerical Study:

- ▶ We now want to study the combination:
Newton's method + inexact steps + CG as linear system solver
for the logistic regression problem (1).
- ▶ Since m and n can be **very large**, this might be an efficient strategy!

A 2D-Example

We generate the two data classes as point clouds in $\mathbb{R}^2$:

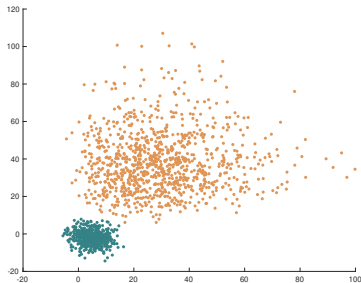
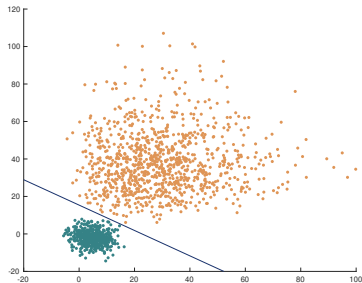
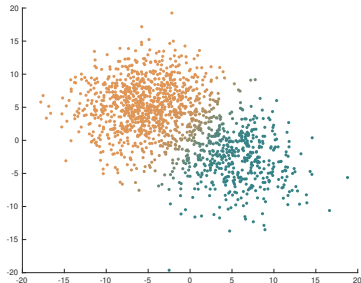
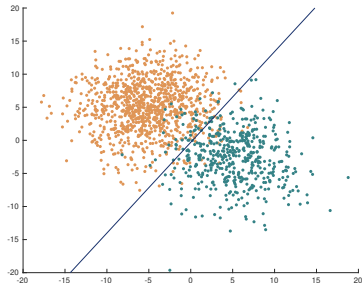
$$\mathbf{A}_1 = 4 \cdot \text{randn}(m_1, 2) + \text{ repmat}([-5, 5], m_1, 1)$$

$$\mathbf{A}_2 = 4 \cdot \text{randn}(m_2, 2) + \text{ repmat}([5, -2.5], m_2, 1),$$

where $m_1 = 1000$, $m_2 = 500$, and $\lambda = 10^{-6}$.

- ▶ We implement the Newton-CG method with $\sigma = 0.5$, $\gamma = 0.1$ (back-tracking parameter), $\mathbf{x}^0 = \mathbf{0}$, $\rho_k = \min\{1, \sqrt{\|\nabla f(\mathbf{x}^k)\|}\}$, and max. CG-iterations `cg_max` = 10 or 20.
- ▶ We compare the obtained result with the solution of an identical support vector machine problem.
- ▶ This is a problem in $\mathbb{R}^3$ ($n = 3$) with $m = 1500$ data points.

SVM vs. Logistic Regression



Real World Examples

data set	data points m	dim. n	comment
mushrooms	8124	112	
gisette	6000	5000	NeurIPS '03 feature sel. challenge
rcv1	20242	47236	text categorization

- ▶ We test the performance of the globalized Newton method and Newton-CG on several real world datasets.
- ▶ The regularization parameter is set to $\lambda = 4/m$; we use MATLAB's backslash operator to solve the Newton equation.

dataset	Global-Newton		Newton-CG	
	iter	time	iter	time
mushrooms	10	0.08 sec	5	0.03 sec
gisette	10	19.86 sec	10	0.64 sec
rcv1	7	365.47 sec	6	0.21 sec

More Examples and Discussion

Discussion:

- The combination:

Newton's method + inexact steps + CG as linear system solver
can be highly efficient for large-scale optimization problems!

- In **logistic regression**, Newton-CG achieves a speed-up of **1800×** compared to a naïve implementation of Newton's method!

dataset	data m	dim. n	Global-Newton		Newton-CG	
			iter	time	iter	time
real-sim	72309	20959	7	558.99 s	11	0.41 s
epsilon	400000	2000	6	1135.71 s	5	88.74 s
news20*	19996	1355192	—	—	4	0.70 s

*From the website <https://www.csie.ntu.edu.tw/~cjlin/libsvm/>

Questions?